A PARADIGM FOR INCORPORATING VISION IN THE ROBOT NAVIGATION FUNCTION *

Vladimir Lumelsky and Tim Skewis
Yale University, Department of Electrical Engineering
New Haven, Connecticut 06520, USA

Abstract

A model of robot navigation is considered whereby the robot is a point automaton whose environment may include unknown obstacles of arbitrary shape. The automaton's input information includes its own and the target point coordinates. It is known that in the framework of this model, assuming the automaton is equipped with the simplest tactile sensing capability for detecting the obstacles, algorithms can be designed that guarantee either reaching the target or proving that it is not reachable. There is an important question whether richer sensing capabilities, such as vision, can be naturally incorporated in the model so as to further improve the performance of the motion planning system (e.g., to shorten the generated paths). This paper presents a paradigm for combining vision with motion planning. It turns out that extensive modifications of the "tactile" algorithms are needed in order to take full advantage of the additional sensing capabilities, while not sacrificing the algorithm convergence. Also, different design principles can be introduced which result in algorithm versions that exhibit different "styles" of behavior and produce different paths, without, in general, being superior to each other.

1. Introduction

Efficient utilization of available information for the purpose of robot navigation presents an important problem in robotics. Assume that the robot is a point automaton (e.g. an autonomous vehicle) traveling along a two-dimensional surface with obstacles; its task is to produce a collision-free path between the starting and the target locations. Current research on robot motion planning revolves around two models that are based on different assumptions about the information available for planning. In the first model, called *path planning with complete information* (another popular term is the *Piano Movers problem*), perfect information about the automaton and the obstacles is assumed. The surfaces of the moving body and of the obstacles must be algebraic; in most works, a stricter requirement of planar surfaces is imposed. Under this model, the whole operation of path planning is a one-time, off-line operation. Given the fact that the amount of information that has to be processed is typically large, obtaining a computationally efficient scheme presents the main difficulty. A good survey of work in this area can be found in [1].

This paper is concerned with the second model, called *path planning with incomplete information*, or path planning with uncertainty. In this model, an element of uncertainty is present, and the missing data is typically provided by some source of local information (e.g., sensory feedback, such as an ultrasound range finder or a vision module). The attractiveness of this model for robotics lies in the possibility to naturally introduce a powerful notion of feedback control, and thus transform the operation of path planning into a continuous, dynamic on-line process. Because little information is being processed at each step, no computational difficulties appear. Also, the model allows one to ease the requirements on the shape and location of the obstacles in the environment (the scene), and even to remove the requirement that the obstacles be stationary.

One important issue in motion planning with incomplete information is the way the sensory information is incorporated in the planning function. One approach in this area calls for separation of the functions of scene reconstruction and motion planning. In such a system, the environment, or a part of it, is first reconstructed based on the sensor data, independent of how the produced information is going to be used later. Then, an algorithm that operates under an assumption of complete information is used for motion planning. This approach gives rise to various techniques of scene reconstruction that are either weakly tied to the task of path planning [2,3,4,5], or are completely independent from it [6,7].

Another approach to sensor-based motion planning calls for an intimate integration of the sensory capability into the planning function. Under this approach, the automaton is deciding at each point of its path what sensory information is required for generating its next step. The approach of *dynamic path planning* [8,9] belongs to this category. It has been shown [8] that within the framework of the model with incomplete information even the simplest tactile sensing capabilities are sufficient for designing path planning algorithms with proven convergence. The approach can be generalized to simple robot arm manipulators of different kinematic configurations [9].

A natural question arises whether the algorithms with incomplete information would exhibit better *path length performance* – that is, generate shorter paths – if a richer source of the on-line input information, such as vision, is available. In other words, a seemingly naive question is being asked: compared, for example, to tactile sensing, does the vision really help in motion planning? in what way and under what conditions?

* Supported in part by the National Science Foundation Grants DMC-8519542 and DMC-8712357, Unimation Inc., and Transitions Research Corp.

This paper presents a paradigm which includes vision in the motion planning function, such that the convergence of the algorithms is preserved and the performance of the algorithms is, in general, improved. It turns out that extensive modifications of the "tactile" algorithms are needed in order to take full advantage of the additional sensing capabilities, while not sacrificing the algorithm convergence. Also, different design principles can be introduced which result in algorithm versions that exhibit different "styles" of behavior and produce different paths, without, in general, being superior to each other. The accepted model and required definitions are presented in Section 2. The basic idea of the algorithms is discussed in Section 3, followed, in Sections 4 and 5, by two algorithms, along with the pertinent analysis. To save space, we omit the relevant proofs.

2. Model

The environment (the scene) is a plane with a set of obstacles and the points Start (S) and Target (T) in it. Each obstacle is a simple closed curve of finite length such that a straight line will cross it only in a finite number of points; a case when the straight line coincides with a finite segment of the obstacle boundary is not a crossing. Obstacles do not touch each other. The environment can contain only a locally finite number of obstacles; this means that any disc of finite radius intersects a finite set of obstacles. Note that the model does not require that the set of obstacles is finite.

The automaton is a point. Its input information includes the coordinates of its current location, C, and the coordinates of the target T. The automaton is capable of moving along a straight line and along the obstacle boundaries. It also has a capability, referred to as *vision*, which allows it to detect an obstacle, if any, and the distance to it, in any direction from point C, within its *field of vision*. The field of vision presents a disc of radius r_v (*radius of vision*) centered at C. A point Q is *visible* if, first, it is located within the field of vision, and second, the straight line segment CQ does not cross any obstacles.

To use its vision, the automaton performs a *scanning* operation of the field of vision during which it *identifies* obstacles, or the lack of thereof, that intersect the whole field of vision or appear in a specific direction. Only visible points or sets of points can be identified during the scanning operation. Thus, the automaton can, for example, identify some intermediate target point that lies within its field of vision and walk toward that point along a straight line. On the other hand, the automaton can use its capability for walking along the obstacle boundary to maneuver around a convex obstacle when the visibility of the obstacle boundary shrinks to zero.

The memory of the automaton is limited to few computer words. For example, it allows the automaton to store coordinates of a few "interesting" points. Because obstacles are of arbitrary shape, storing them requires infinite memory. Thus, the automaton is not able of recognizing an obstacle that it passed before.

A desirable path to T, called the *main line* or *M-line*, is introduced as a straight line segment that connects S and T. At the moment i, at its current position C_i on the path, the automaton scans its current field of vision, and uses the collected information to define its *next intermediate target*, or T_i. Then, the automaton makes a little step in the direction of T_i, and the process repeats. In the algorithms described below, every T_i lies

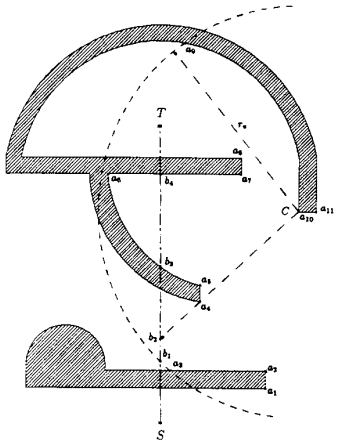

Figure 1: At its current location C, the robot will see obstacles within its radius of vision r_v, such as the segments of obstacle boundaries $a_1a_2a_3$, $a_4a_5a_6a_7a_8$, $a_9a_{10}a_{11}$. It will also conclude that the segments b_1b_2 and b_3b_4 of the M-line are "visible".

either on the M-line or on an obstacle boundary. For a segment of the path where T_i moves along the M-line, the first defined T_i that lies at the intersection between the M-line and an obstacle is a special point called the *hit point*, H. For a segment of the path where T_i moves along an obstacle boundary, the first defined T_i that lies at the M-line is a special point called the *leave point*, L. The main difference between the various algorithms with vision is in how they define T_i. Naturally, the current T_i is always at a distance from the automaton not more than r_v.

While scanning the field of vision, the automaton may be detecting some contiguous sets of visible points – for example, a segment of the obstacle boundary. A point Q is *contiguous* to another point S *over the set* $\{P\}$, if (i) $S \in \{P\}$, (ii) Q and $\{P\}$ are visible, and (iii) Q can be continuously connected with S using only points of $\{P\}$. A *set is contiguous* if any pair of its points are contiguous over the set.

A *local direction* is a once-and-for-all determined direction for passing around an obstacle; facing the obstacle, it can be either left or right. Because of incomplete information, neither local direction can be judged better than the other. For the sake of clarity, assume that the local direction is always *left*. Below, $d(A, B)$ is the Euclidean distance between the points A and B in the scene.

The M-line divides the environment into two half planes. The half plane that lies to the local direction side of the M-line is called the *main semi-plane*, and the other half plane is called the *secondary semi-plane*. Thus, if the local direction is "left", then the left half plane, looking from S toward T, is the main semi-plane.

An example in Figure 1 demonstrates the defined terms. Here, the straight line segment ST is the M-line; the current location of the automaton, C, is in the secondary (right) semi-plane; its field of vision is of radius r_v. If, while standing at C, the automaton performs the scanning operation, it will identify contiguous segments of the obstacle boundaries, $a_1a_2a_3$, $a_4a_5a_6a_7a_8$, and $a_9a_{10}a_{11}$, and contiguous segments of the M-line, b_1b_2 and b_3b_4.

3. Basic methodology

As an example, we incorporate the vision capability into one of the algorithms, called *Bug2*, described in [8]. The algorithm is based on the model above, except that its sensing capability is limited to tactile sensing; this can be interpreted as zero vision, $r_v = 0$. Under the algorithm Bug2, the automaton can meet the same obstacle more than once, and it has no way of distinguishing between different obstacles. The subscript j will indicate the j-th occurrence of the hit or leave points on the same or on a different obstacle. Initially, $j = 1$; L_0=Start. The algorithm consists of the following steps (follow Figure 2).

1. From point L_{j-1}, move along the straight line segment ST until one of the following occurs:
   (a) T is reached. The procedure stops.
   (b) An obstacle is encountered and a hit point, H_j, is defined. Go to Step 2.
2. Using the accepted local direction, follow the obstacle boundary until one of the following occurs:
   (a) T is reached. The procedure stops.
   (b) The line segment ST is met at a point Q such that the distance from Q to T is less than that from H_j to T, and the line segment QT does not cross the current obstacle at the point Q. Define the leave point $L_j = Q$. Set $j = j + 1$. Go to Step 1.
   (c) The automaton returns to H_j and thus completes a closed curve (the obstacle boundary) without having defined tne next hit point, H_{j+1}. The target cannot be reached. The procedure stops.

Because we want to preserve convergence of the path planning procedures, introducing vision in an algorithm, such as Bug2, cannot be done in a direct way. For example, it can be shown that simply walking to the farthest visible "corner" of an obstacle that lies in the "right" direction can ruin the convergence (for more detail, see [9]).

Since the procedure Bug2 is known to converge, one way of using vision is to organize the algorithm such that at each step of its path the automaton "mentally" reconstructs in its current field of vision the segment of the path that would be produced by Bug2, makes the farthest point of that segment its intermediate target, and proceeds directly toward that target. As it turns out, deciding whether a given point lies on the path that would be generated by Bug2 (or, for short, whether a given point is a *Bug2 point*) is not a trivial task. The resulting procedure is called below VisBug-21 (where the first numeral, "2", refers to Bug2).

A different procedure, also based on the mechanism of the Bug2 algorithm, can be designed that behaves more opportunistically than VisBug-21: namely, instead of using the Bug2 path for generating its own path, the procedure can deviate from what is dictated by the Bug2 path, in order to take advantage of the opportunities that look more promising (as long as convergence is preserved). This procedure is called below VisBug-22. As one will see in the following sections, under the same initial conditions both procedures, VisBug-21 and VisBug-22, can produce quite different paths. Furthermore, depending on the environment, one procedure may be more efficient (that is, produce

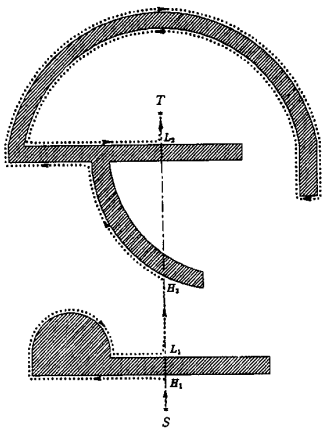

Figure 2: Environment 1: path generated by the algorithm Bug2.

a shorter path) than the other, and so both present viable options. Both algorithms include a test for target reachability that is based on the following necessary and sufficient condition: if, after having defined the last hit point as its intermediate target, the automaton returns to it before it defines the next hit point, then either the automaton or the target point is *trapped* and hence the target is not reachable (for more detail, see [8]). Both allow one to incorporate some features of practical value, such as a *safety margin* which presents a minimum distance from an obstacle that the automaton is required to maintain.

4. Algoritnm VisBug-21

Algorithm.

The algorithm consists of the *Main body* that does the proper motion planning along the path, and a procedure called *Compute T_i-21*, which performs the test for target reachability and produces the next intermediate target T_i for a given current position of the automaton C. The main body is designed such that in most cases the operation is confined to its step S1 (see below). The second step, S2, is executed only in those cases when the automaton is moving along a (locally) convex boundary of an obstacle and so it cannot use its vision for defining the next intermediate target T_i. For the reasons that will become clear in the ensuing analysis, the algorithm distinguishes the case when the point T_i lies in the main semi-plane from the case when T_i lies in the secondary semi-plane. Initially, $C = T_i = S$.

Main body. This procedure is executed at each point of the continuous path. It consists of the following steps:

- S1: Move towards T_i while executing *Compute T_i-21* and performing the following test:
  If $C = T$ the procedure stops.
  Else if the target is unreachable the procedure stops.
  Else if $C = T_i$ go to step S2.
- S2: Move along the obstacle boundary while executing *Compute T_i-21* and performing the following test:
  If $C = T$ the procedure stops.
  Else if the target is unreachable the procedure stops.
  Else if $C \neq T_i$ go to step S1.

Procedure *Compute T_i-21.* The procedure consists of the following steps:

- Step 1: If T is visible then define $T_i = T$; procedure stops.
  Else if T_i is on an obstacle boundary go to step 3.
  Else go to step 2.
- Step 2: Define point Q as the endpoint of the maximum length contiguous segment of the M-line, T_iQ, extending from T_i in the direction of T.
  If an obstacle has been identified crossing the M-line at point Q then define a hit point, $H = Q$; assign $X = Q$, define $T_i = Q$; go to step 3.
  Else define $T_i = Q$; go to step 4.
- Step 3: Define point Q as the endpoint of the maximum length contiguous segment of the obstacle boundary, T_iQ, extending from T_i in the local direction.
  If the obstacle has been identified crossing the M-line at a point $P \in T_iQ$, $d(P,T) < d(H,T)$, then assign $X = P$ and if in addition the line segment PT does not cross the obstacle at P then define a leave point, $L = P$, define $T_i = P$, and go to step 2.
  If the lastly defined hit point, H, is again identified and $H \in T_iQ$ then the target is not reachable; procedure stops.
  Else define $T_i = Q$; go to step 4.
- Step 4: If T_i is on the M-line define $Q = T_i$, otherwise define $Q = X$.
  If points, $\{P\}$, on the M-line are identified such that $d(S',T) < d(Q,T)$, $S' \in \{P\}$, and C is in the main semi-plane then find the point $S' \in \{P\}$ that produces the shortest distance $d(S',T)$; define $T_i = S'$; go to step 2.
  Else procedure stops.

In simple terms, the procedure *Compute T_i-21* operates as follows. Step 1 is executed at the (last) stage when the target T becomes visible (e.g., at point A, Figure 4). A special case, in which points of the M-line noncontiguous to the previously considered sets of points are tested as candidates for the next intermediate target T_i, is handled in Step 4. All the remaining situations relate to choosing the next T_i among the points of the Bug2 path contiguous to the previously defined T_i; these are treated in Steps 2 and 3. Specifically, in Step 2 candidate points along the M-line are processed, and hit points are defined. In Step 3, candidate points along obstacle boundaries are processed, and leave points are defined. The test for target reachability is also performed in Step 3. It is conceivable that, for a given current location C of the automaton, the procedure will execute, perhaps even more than once, some combination of Steps 2, 3, and 4. While doing that, contiguous and noncontiguous segments of the Bug2 path along the M-line and along obstacle boundaries are considered, before the next intermediate target T_i is defined and the automaton makes a physical step towards T_i.

Analysis.

In Figures 3 and 4, the performance of the algorithm VisBug-21 is demonstrated for two different values of the radius of vision r_v (compare with the performance of the algorithm Bug2 in the same environment, Figure 2). Since the path generated by the algorithm can diverge significantly from the corresponding path that would be produced under the same conditions by the algorithm Bug2, it is to be shown that the path length performance

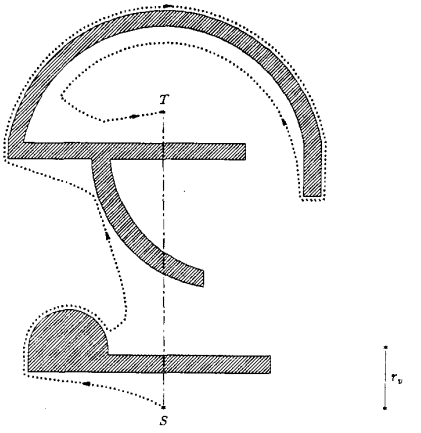

Figure 3: Environment 1: path generated by the algorithms VisBug-21 or VisBug-22; the radius of vision is r_v.

of VisBug-21 is never worse than that of Bug2. Such an expected behavior is assured by the following lemma.

Lemma 1. *For a given environment and a given set of start and target points, the path produced by the algorithm VisBug-21 is never longer than that produced by the algorithm Bug2.*

One can also see from Figure 4 that when r_v goes to infinity, the algorithm VisBug-21 will generate locally optimal paths, in the following sense. Take two obstacles or parts of the same obstacle, k and $k+1$, that are visited by the automaton, in this order. During the automaton's interaction with (i.e., passing around) the obstacle k, once the obstacle $k+1$ is identified as the next intermediate target, the area between k and $k+1$ will be covered along the straight line which presents the locally shortest path.

To define its next intermediate target, T_i, the algorithm VisBug-21, as presented above, sometimes uses points on the M-line that are not necessarily contiguous to the previous intermediate target T_{i-1}. To assure convergence, it is important to

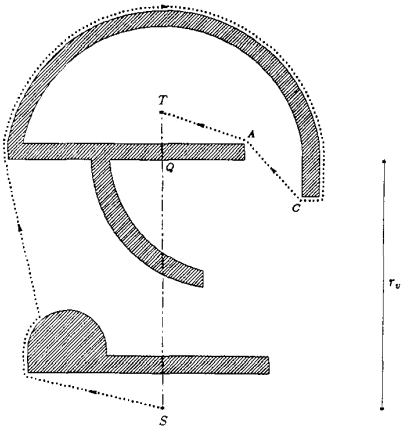

Figure 4: Environment 1: path generated by VisBug-21; the radius of vision r_v is larger than that in Figure 3.

show that in such cases the considered candidate points on the M-line do indeed lie on the Bug2 path.

Within the context of the algorithm VisBug-21, consider some current location C of the automaton along the VisBug-21 path. Also consider some visible point Q on the M-line such that $d(Q,T) < d(T_{i-1},T)$ if T_{i-1} is on the M-line, or such that $d(Q,T) < d(X,T)$ if T_{i-1} is on an obstacle boundary; here, X is defined as in Steps 2 and 3 of the procedure *Compute T_i-21*. Point Q is said to be *further along the Bug2 path* than point T_{i-1} if, first, it lies on the Bug2 path, and second, the length of the Bug2 path segment between Q and T is shorter than that between T_{i-1} and T. Then, the following statement will be useful for proving the convergence of the VisBug-21 algorithm.

Lemma 2. *For the point Q to be further along the Bug2 path than T_{i-1}, it is sufficient if C lies in the main semi-plane.*

In other words, if the automaton is currently located in the secondary semi-plane, then some point that lies on the M-line and seems otherwise a good candidate for the next intermediate target T_i, may or may not lie on the Bug2 path. Thus, such a point should not be even considered. Such an example is shown in Figure 4 where, while at the location C, the automaton will reject the seemingly attractive point Q (Step 2 of the algorithm) because it does not lie on the Bug2 path.

The convergence of the algorithm VisBug-21 follows from the fact that the generated path is tied – either explicitly or implicitly, through the mechanism analysed in Lemma 2 – to the Bug2 path for which convergence has been already established.

5. Algorithm VisBug-22

Algorithm.

The structure of this algorithm is very similar to VisBug-21, except the automaton does not try to ensure that all the intermediate targets T_i lie on the Bug2 path. Instead, it tries to define its intermediate targets along the M-line as close to the target T as possible. This results in a different behavior compared to VisBug-21; also, the convergence of VisBug-22 comes from a source other than the convergence of the algorithm VisBug-21.

Consider some environment with the given points S and T. Consider the third point, S', that lies on the M-line somewhere between S and T. The path produced by the algorithm Bug2 between S and T is said to be the *Bug2 path*. A *quasi-Bug2 path segment* is a contiguous segment that starts at S' and produces a part of the path that the Bug2 algorithm would have generated if the points S' and T were its starting and target points, respectively. As the point S' needs not be on the Bug2 path, the quasi-Bug2 path segment needs not be a segment of the Bug2 path.

The algorithm VisBug-22 will identify points along the Bug2 path or a quasi-Bug2 path segment, until a better, in terms of its proximity to T, point on the M-line is identified. Then, this point, S', becomes the starting point of another quasi-Bug2 path segment, and the process repeats. As a result, unlike the algorithms Bug2 and VisBug-21, where each defined hit point has its matching leave point, in VisBug-22 no such matching necessarily occurs. In order to assure convergence, the point S' must satisfy certain requirements in order to be chosen as the starting point of the next quasi-Bug2 path segment.

The algorithm consists of the *Main body*, which is identical to the main body of the algorithm VisBug-21, and a procedure called *Compute T_i-22*, which performs the test for target reachability and produces the next intermediate target T_i for a given current position of the automaton C. Initially, $C = S = T_i$.

Procedure *Compute T_i-22* of the algorithm VisBug-22. The procedure consists of the following steps:

- Step 1: If T is visible then define $T_i = T$; procedure stops.
  Else if T_i is on an obstacle boundary go to step 3.
  Else go to step 2.
- Step 2: Define point Q as the endpoint of the maximum length contiguous segment of the M-line, T_iQ, extending from T_i in the direction of T.
  If an obstacle has been identified crossing the M-line at point Q then define a hit point, $H = Q$; define $T_i = Q$; go to step 3.
  Else define $T_i = Q$; go to step 4.
- Step 3: Define point Q as the endpoint of the maximum length contiguous segment of the obstacle boundary, T_iQ, extending from T_i in the local direction.
  If the obstacle has been identified crossing the M-line at a point $P \in T_iQ$, $d(P,T) < d(H,T)$ and the line segment PT does not cross the obstacle at P then define a leave point, $L = P$, define $T_i = P$, and go to step 2.
  If the lastly defined hit point, H, is again identified and $H \in T_iQ$ then the target is not reachable; procedure stops.
  Else define $T_i = Q$; go to step 4.
- Step 4: If T_i is on the M-line define $Q = T_i$, otherwise define $Q = H$.
  If points, $\{P\}$, on the M-line are identified such that $d(S',T) < d(Q,T)$, $S' \in \{P\}$ then find the point $S' \in \{P\}$ that produces the shortest distance $d(S',T)$;
  define $T_i = S'$; go to step 2.
  Else procedure stops.

Analysis.

The performance of the algorithm VisBug-22 is demonstrated in Figures 3 and 5, where the same values of the radius of vision r_v as in the examples for VisBug-21, Figures 3 and 4, are used; compare with the performance of the algorithm Bug2 in the same environment, Figure 2. Note that the performance of both algorithms VisBug-21 and -22 can sometimes be identical (Figure 3). In general, neither algorithm is superior to the other: for example, in Figures 4 and 5, the performance of VisBug-21 is better than that of VisBug-22, whereas the opposite is true in the examples shown in Figures 6 and 7. Although typically VisBug-22 will generate a path significantly better than that generated by the "tactile" algorithm Bug2, this cannot be guaranteed. In other words, no lemma similar to Lemma 1 can be shown to hold for the algorithm VisBug-22. In fact, Figure 5 demonstrates that under the same initial conditions VisBug-22 can generate a longer path than the algorithm Bug2.

The convergence of the algorithm VisBug-22 follows from the fact that all the starting points, S', of the successive quasi-Bug2 path segments lie on the M-line and they are organized in such a way as to produce a finite sequence of distances shrinking to T:

$$d(S'_1,T) > d(S'_2,T) > d(S'_3,T) > ...,$$

where points S' are numbered in the order of their appearance.

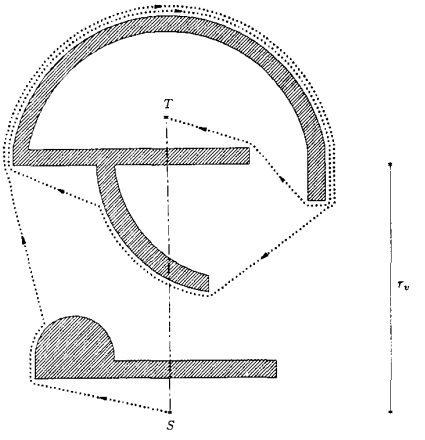

Figure 5: Environment 1: path generated by VisBug-22: the radius of vision r_v is larger than that in Figure 3 (and equal to that in Figure 4).

6. Conclusion

A paradigm has been presented for incorporating the vision information into the robot path planning operation. The key issues addressed in this paradigm are, first, how to use local sensor information in order to guarantee reaching a goal (if it is reachable) in an unknown environment, and second, how to organize in the most efficient way the feedback interaction between the subsystems responsible for sensor data gathering and path planning. Along these lines, two algorithms were described that not only efficiently use vision information but actually determine what kind of vision information should be gathered at each step. For example, if the vision information is provided by a sequential scanning range finder, then the robot will be requested the range information in a specific direction, and will not have to scan the whole 360° circumference.

This work raises a host of theoretical and practical issues on the relationship between the amount of sensor information and the efficiency of algorithms in the problem of motion planning with uncertainty. For example, it has been shown (Lemma 1 above) that, for a given environment and a given set of start and target positions, the algorithm VisBug-21 will never generate a path longer than that of the "tactile" algorithm Bug2. On the other hand, surprisingly, quite natural-looking examples can be designed where a better vision (i.e., a larger radius of vision r_v) produces a path longer than that produced by the same algorithm and under the same conditions except with a more inferior vision (a smaller r_v). Also, it is difficult to do direct comparison between the two described algorithms, VisBug-21 and VisBug-22. Depending on the environment, each of them may lose to the other or outperform the other.

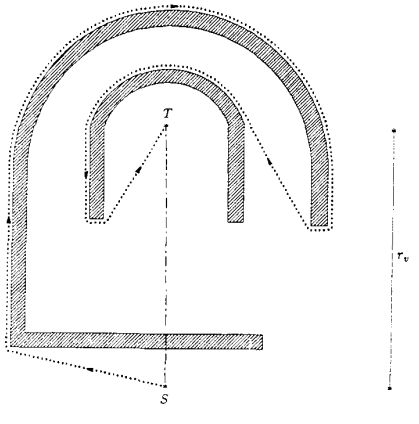

Figure 6: Environment 2: path generated by VisBug-21.

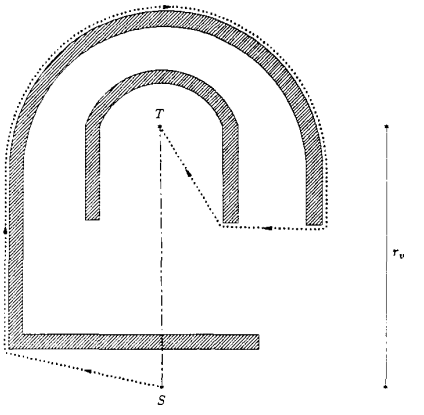

Figure 7: Environment 2: path generated by VisBug-22.

References

[1] C. Yap, Algorithmic Motion Planning. In *Advances in Robotics: Algorithmic and Geometric Aspects*, (J. Schwartz and C. Yap, Eds.). Hillsdale, NJ, Erlbaum, 1987.

[2] M. Hebert, Outdoor Scene Analysis Using Range Data. *Proc. 1986 IEEE International Conf. on Robotics and Automation*, San Francisco, April 1986.

[3] L. Matthies, S. Shafer, Error Modeling in Stereo Navigation. IEEE Journal of Robotics and Automation, June 1987.

[4] A. Elfes, Sonar Based Real-World Mapping and Navigation. IEEE Journal of Robotics and Automation, June 1987.

[5] D. Kriegman, E. Triendl, T. Binford, A Mobile Robot: Sensing, Planning and Locomotion. *Proc. 1987 IEEE International Conf. on Robotics and Automation*, Raleigh, N.C., April 1987.

[6] N. Rao, S. Iyengar, C. Jorgensen, C. Weisbin, On Terrain Acquisition by a Finite-Sized Mobile Robot in Plane. *Proc. 1987 IEEE International Conf. on Robotics and Automation*, Raleigh, N.C., April 1987.

[7] R. Cole and C. Yap, Shape from Probing. New York University, Courant Institute, Technical Report No. 104, December 1983.

[8] V. Lumelsky, A. Stepanov, Dynamic Path Planning for a Mobile Automaton with Limited Information on the Environment. *IEEE Transactions on Automatic Control*, Vol.AC-31, No.11, November 1986.

[9] V. Lumelsky, Algorithmic and Complexity Issues of Robot Motion in an Uncertain Environment. *Journal of Complexity*, September 1987.